

Bash script Tasks

Github Repository url: <https://github.com/vamshikumar-a/bash-scripting.git>

Task 1: Create a bash script to check if a directory is available or not.

Objective:

Create a Bash script to check whether a directory is available.

Purpose:

To verify directory availability before performing operations.

Implementation:

1. Created check_directory.bash.
2. Used the -d condition to check the directory.
3. Added an if-else statement to display the result.
4. Made the script executable using chmod +x.
5. Executed the script and verified the output.

```
[root@ip-172-31-26-233 scripts]# mkdir sample
[root@ip-172-31-26-233 scripts]# vi check_directory.bash
[root@ip-172-31-26-233 scripts]# chmod 755 check_directory.bash
[root@ip-172-31-26-233 scripts]# ./check_directory.bash
Enter directory path: /opt/home/scripts/sample
Directory exists.
[root@ip-172-31-26-233 scripts]# cat check_directory.bash
#!/bin/bash
# Purpose: Checking for directory exists or not
# Date: 15/08/2026
# Modification: 15/08/2026
# Owner= vamshi

#!/bin/bash

read -p "Enter directory path: " dir

if [ -d "$dir" ]
then
    echo "Directory exists."
else
    echo "Directory does not exist."
fi
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully checked directory availability using a Bash script.

Task 2: Create a bash script to create multiple files.

Objective:

Create multiple files using a Bash script.

Purpose:

To automate file creation.

Implementation:

1. Created create_files.bash.
2. Used a for loop to create multiple files.
3. Used the touch command to create the files.
4. Made the script executable using chmod +x.
5. Executed the script.
6. Verified the files using ls.

```
[root@ip-172-31-26-233 scripts]# nano create_files.bash
[root@ip-172-31-26-233 scripts]# cat create_files.bash
#!/bin/bash
# Purpose: Creating multiple files
#Date: 15/08/2026
#Modification: 15/08/2026
#Owner= vamshi

for i in {1..5}
do
    touch "file$i.txt"
done

echo "5 files created successfully."

[root@ip-172-31-26-233 scripts]# chmod 755 create_files.bash
[root@ip-172-31-26-233 scripts]# ./create_files.bash
5 files created successfully.
[root@ip-172-31-26-233 scripts]# ls
check_directory.bash  create_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  sample
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully created multiple files using a Bash script.

Task 3: Create a bash script to take a backup of a directory.

Objective:

Create a Bash script to back up a directory.

Purpose:

To automate directory backup and protect important files.

Implementation:

1. Created back_directory.bash.
2. Specified the source and backup directories.
3. Created the backup directory using mkdir -p.
4. Used tar -czvf to create a compressed backup.
5. Added the date and time to the backup filename.
6. Made the script executable using chmod +x.

7. Executed the script and verified the backup file.

```
[root@ip-172-31-26-233 scripts]# nano backup_directory.bash
[root@ip-172-31-26-233 scripts]# ls
backup_directory.bash  check_directory.bash  create_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  sample
[root@ip-172-31-26-233 scripts]# cat backup_directory.bash
#!/bin/bash
# Purpose: Taking backup of directory
# Date: 15/08/2026
# Modification: 15/08/2026
# Owner= vamshi

SOURCE="/opt/home/scripts/sample"
BACKUP_DIR="/opt/home/scripts"

mkdir -p "$BACKUP_DIR"

tar -czvf "$BACKUP_DIR/sample-$(date +%Y%m%d-%H%M%S).tar.gz" "$SOURCE"

echo "Backup completed successfully."
echo
ls
[root@ip-172-31-26-233 scripts]# bash backup_directory.bash
tar: Removing leading '/' from member names
/opt/home/scripts/sample/
/opt/home/scripts/sample/test
Backup completed successfully.

backup_directory.bash  check_directory.bash  create_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  sample  sample-20260815-102059.tar.gz
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully created a compressed backup of the directory using Bash scripting.

Task 4: Create a bash script to install Nginx on an EC2 server.

Objective:

Create a Bash script to install and configure Nginx on an EC2 server.

Purpose:

To automate Nginx installation and service management.

Implementation

1. Created install_nginx.sh.
2. Updated the EC2 server packages.
3. Installed Nginx using the package manager.
4. Started the Nginx service using systemctl.
5. Enabled Nginx to start automatically.
6. Verified the Nginx service status.

```
[root@ip-172-31-26-233 scripts]# nano install_nginx.bash
[root@ip-172-31-26-233 scripts]# cat install_nginx.bash
#!/bin/bash
# Purpose: installing nginx and starting server
# Date: 15/08/2026
# Modification: 15/08/2026
# Owner= vamshi

echo "Installing Nginx..."
yum install nginx -y
if [ $? -eq 0 ]
then
    echo "Nginx installation successful."
else
    echo "Nginx installation failed."
    exit 1
fi

echo "Starting Nginx..."
systemctl start nginx
if [ $? -eq 0 ]
then
    echo "Nginx started successfully."
else
    echo "Failed to start Nginx."
    exit 1
fi
```

```
root@ip-172-31-26-233:/opt/home/scripts# systemctl status nginx
systemctl status nginx
if [ $? -eq 0 ]
then
    echo "Nginx running successfully."
else
    echo "Failed to run Nginx."
fi
exit 1
echo "Nginx installation and started successfully."
```

```
[root@ip-172-31-26-233 scripts]# bash install_nginx.bash
Installing Nginx...
Last metadata expiration check: 2:48:15 ago on Sat Aug 15 07:47:17 2026.
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
nginx x86_64 1:1.30.4-1.amzn2023.0.1 amazonlinux 34 k
Installing dependencies:
generic-logos-httpd noarch 18.0.0-12.amzn2023.0.3 amazonlinux 19 k
gperftools-libs x86_64 2.9.1-1.amzn2023.0.3 amazonlinux 308 k
t1bunwind x86_64 1.4.0-5.amzn2023.0.3 amazonlinux 66 k
nginx-core x86_64 1:1.30.4-1.amzn2023.0.1 amazonlinux 231 k
nginx-filesystem noarch 1:1.30.4-1.amzn2023.0.1 amazonlinux 10 k
nginx-mimetypes noarch 2.1.49-3.amzn2023.0.3 amazonlinux 21 k
Transaction Summary
Install 7 Packages
Total download size: 1.1 M
Installed size: 3.9 M
Downloading Packages:
(1/7): gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm 8.3 MB/s | 308 kB 00:00
```

```
root@ip-172-31-26-233:/opt/home/scripts#
Installing : nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch 1/7
Installing : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch 2/7
Installing : t1bunwind-1.4.0-5.amzn2023.0.3.x86_64 3/7
Installing : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64 4/7
Installing : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64 5/7
Installing : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 6/7
Installing : nginx-1:1.30.4-1.amzn2023.0.1.x86_64 7/7
Running scriptlet: nginx-1:1.30.4-1.amzn2023.0.1.x86_64 7/7
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 1/7
Verifying : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64 2/7
Verifying : t1bunwind-1.4.0-5.amzn2023.0.3.x86_64 3/7
Verifying : nginx-1:1.30.4-1.amzn2023.0.1.x86_64 4/7
Verifying : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64 5/7
Verifying : nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch 6/7
Verifying : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch 7/7

Installed:
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64 t1bunwind-1.4.0-5.amzn2023.0.3.x86_64
nginx-1:1.30.4-1.amzn2023.0.1.x86_64 nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64 nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch
nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
Nginx installation successful.
Starting Nginx...
Nginx started successfully.
Checking status Nginx...
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-08-15 10:35:34 UTC; 13ms ago
     Process: 33485 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (Code=exited, status=0/SUCCESS)
     Process: 33486 ExecStartPre=/usr/sbin/nginx -t (Code=exited, status=0/SUCCESS)
     Process: 33487 ExecStart=/usr/sbin/nginx (Code=exited, status=0/SUCCESS)
    Main PID: 33488 (nginx)
       Tasks: 3 (limit: 1059)
         CPU: 4.0M
        Memory: 4.0M
          CPU: 56ms
      CGroup: /system.slice/nginx.service
              └─33488 "nginx: master process /usr/sbin/nginx"
                └─33489 "nginx: worker process"
                  └─33490 "nginx: worker process"

Aug 15 10:35:34 ip-172-31-26-233.ec2.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Aug 15 10:35:34 ip-172-31-26-233.ec2.internal nginx[33486]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Aug 15 10:35:34 ip-172-31-26-233.ec2.internal nginx[33486]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Aug 15 10:35:34 ip-172-31-26-233.ec2.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
Nginx running successfully.
Nginx installation and started successfully.
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully automated Nginx installation and configuration using a Bash script.

Task 5: Create a bash script to install Apache Tomcat on an EC2 server.

Objective:

Create a Bash script to install and configure Apache Tomcat.

Purpose:

To automate Java and Tomcat installation for hosting Java applications.

Implementation:

1. Created install_tomcat.sh.
2. Updated the EC2 server packages.
3. Installed Java using the package manager.
4. Downloaded the Apache Tomcat archive.
5. Extracted Tomcat under /opt.

6. Renamed the extracted directory to tomcat.

7. Started Tomcat using startup.sh.

```
root@ip-172-31-26-233:/opt/home/scripts
[root@ip-172-31-26-233 scripts]# nano install_tomcat.bash
# Purpose: installing and starting tomcat
# Date: 15/08/2026
# Modification: 15/08/2026
# Owner: vamsi

echo "Installing Java..."
yum install java -y
if [ $? -eq 0 ]
then
    echo "Java installation successful."
else
    echo "Java installation failed."
    exit 1
fi

echo "Checking Java version..."
java -version
if [ $? -eq 0 ]
then
    echo "Java is working successfully."
else
    echo "Java is not working."
    exit 1
fi

echo "Downloading Apache Tomcat..."
cd /opt
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.48/bin/apache-tomcat-10.1.48.tar.gz
if [ $? -eq 0 ]
then
    echo "Tomcat download successful."
else
    echo "Tomcat download failed."
    exit 1
fi

echo "Extracting Tomcat..."
```

```
root@ip-172-31-26-233:/opt/home/scripts
echo "Downloading Apache Tomcat..."
cd /opt
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.48/bin/apache-tomcat-10.1.48.tar.gz
if [ $? -eq 0 ]
then
    echo "Tomcat download successful."
else
    echo "Tomcat download failed."
    exit 1
fi

echo "Extracting Tomcat..."
tar -xvzf apache-tomcat-10.1.48.tar.gz
if [ $? -eq 0 ]
then
    echo "Tomcat extraction successful."
else
    echo "Tomcat extraction failed."
    exit 1
fi

echo "Renaming Tomcat directory..."
mv apache-tomcat-10.1.48 tomcat
echo "Setting permissions..."
chmod 755 /opt/tomcat/bin/*.sh
echo "Starting Tomcat..."
/opt/tomcat/bin/startup.sh
if [ $? -eq 0 ]
then
    echo "Tomcat started successfully."
else
    echo "Failed to start Tomcat."
    exit 1
fi

echo "Tomcat installation completed successfully."
[root@ip-172-31-26-233 scripts]#
```

```
[root@ip-172-31-26-233 scripts]# vi install_tomcat.bash
[root@ip-172-31-26-233 scripts]# ./install_tomcat.bash
Installing Java...
Last metadata expiration check: 1 day, 0:30:01 ago on Sat Aug 15 07:47:17 2026.
Package java-26-amazon-corretto-1:26.0.2+10-1.amzn2023.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
Java installation successful.
Checking Java version...
openjdk version "26.0.2" 2026-07-21
OpenJDK Runtime Environment Corretto-26.0.2.10.1 (build 26.0.2+10-FR)
OpenJDK 64-Bit Server VM Corretto-26.0.2.10.1 (build 26.0.2+10-FR, mixed mode, sharing)
Java is working successfully.
Downloading Apache Tomcat...
--2026-08-16 08:17:19-- https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14400273 (14M) [application/x-gzip]
Saving to: 'apache-tomcat-10.1.57.tar.gz'

apache-tomcat-10.1.57.tar.gz      100%[=====]
2026-08-16 08:17:19 (441 MB/s) - 'apache-tomcat-10.1.57.tar.gz' saved [14400273/14400273]

Tomcat download successful.
Extracting Tomcat...
apache-tomcat-10.1.57/conf/
apache-tomcat-10.1.57/conf/catalina.policy
apache-tomcat-10.1.57/conf/catalina.properties
apache-tomcat-10.1.57/conf/context.xml
apache-tomcat-10.1.57/conf/jaspic-providers.xml
apache-tomcat-10.1.57/conf/jaspic-providers.xsd
apache-tomcat-10.1.57/conf/logging.properties
apache-tomcat-10.1.57/conf/server.xml
apache-tomcat-10.1.57/conf/tomcat-users.xml
apache-tomcat-10.1.57/conf/tomcat-users.xsd
apache-tomcat-10.1.57/conf/web.xml
```

```
root@ip-172-31-26-233:/opt/home/scripts
apache-tomcat-10.1.57/webapps/host-manager/css/manager.css
apache-tomcat-10.1.57/webapps/host-manager/images/asf-logo.svg
apache-tomcat-10.1.57/webapps/host-manager/images/favicon.ico
apache-tomcat-10.1.57/webapps/host-manager/images/tomcat.svg
apache-tomcat-10.1.57/webapps/host-manager/index.jsp
apache-tomcat-10.1.57/webapps/manager/META-INF/context.xml
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/401.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/403.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/404.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/connectorCerts.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/connectorCiphers.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/connectorTrustedCerts.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/sessionDetail.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/jsp/sessionList.jsp
apache-tomcat-10.1.57/webapps/manager/WEB-INF/web.xml
apache-tomcat-10.1.57/webapps/manager/css/manager.css
apache-tomcat-10.1.57/webapps/manager/images/asf-logo.svg
apache-tomcat-10.1.57/webapps/manager/images/favicon.ico
apache-tomcat-10.1.57/webapps/manager/images/tomcat.svg
apache-tomcat-10.1.57/webapps/manager/index.jsp
apache-tomcat-10.1.57/webapps/manager/status.xsd
apache-tomcat-10.1.57/webapps/manager/xform.xsl
apache-tomcat-10.1.57/bin/catalina.sh
apache-tomcat-10.1.57/bin/ciphers.sh
apache-tomcat-10.1.57/bin/configtest.sh
apache-tomcat-10.1.57/bin/dsmon.sh
apache-tomcat-10.1.57/bin/digest.sh
apache-tomcat-10.1.57/bin/makebase.sh
apache-tomcat-10.1.57/bin/migrate.sh
apache-tomcat-10.1.57/bin/setclasspath.sh
apache-tomcat-10.1.57/bin/shutdown.sh
apache-tomcat-10.1.57/bin/startup.sh
apache-tomcat-10.1.57/bin/tool-wrapper.sh
apache-tomcat-10.1.57/bin/version.sh
Tomcat extraction successful.
Renaming Tomcat directory...
Setting permissions...
Starting Tomcat
Using CATALINA_BASE: /opt/tomcat
Using CATALINA_HOME: /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME: /usr
Using CLASSPATH: /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Tomcat started.
Tomcat started successfully.
Tomcat installation completed successfully.
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully automated Apache Tomcat installation and startup using a Bash script.

Task 6: Create a bash script to check if the Nginx service is running, if not running then script should start the service.

Objective:

Create a Bash script to check whether Nginx is running and start it if it is stopped.

Purpose:

To automatically monitor and start the Nginx service when required.

Implementation:

1. Created check_nginx.bash.
2. Used systemctl is-active nginx to check the service status.
3. Used an if-else condition to verify whether Nginx is running.
4. Started Nginx using systemctl start nginx if it was not running.
5. Made the script executable using chmod +x.
6. Executed the script and verified the Nginx status.

```

[root@ip-172-31-26-233 scripts]# nano check_nginx.bash
[root@ip-172-31-26-233 scripts]# cat check_nginx.bash
#!/bin/bash
# Purpose: Checking for running status of nginx
#Date: 16/08/2026
#Modification: 16/08/2026
#Owner= vamshi

echo "Checking Nginx service status..."

if systemctl is-active --quiet nginx
then
    echo "Nginx service is running."
else
    echo "Nginx service is not running."
    echo "Starting Nginx service..."

    systemctl start nginx

    if [ $? -eq 0 ]
    then
        echo "Nginx service started successfully."
    else
        echo "Failed to start Nginx service."
        exit 1
    fi
fi

echo "Script execution completed."
[root@ip-172-31-26-233 scripts]# chmod 755 check_nginx.bash
[root@ip-172-31-26-233 scripts]# ./check_nginx.bash
Checking Nginx service status...
Nginx service is running.
Script execution completed.
[root@ip-172-31-26-233 scripts]# |

```

Conclusion:

Successfully created a Bash script to monitor and start Nginx automatically.

Task 7: Create a bash script for a calculator.

Objective:

Create a Bash script to perform basic arithmetic calculations.

Purpose:

To perform addition, subtraction, multiplication, and division using Bash.

Implementation:

1. Created calculator.bash.
2. Used read to accept two numbers.
3. Used read to accept the arithmetic operator.
4. Used a case statement to perform the selected operation.
5. Displayed the calculated result.
6. Made the script executable using chmod +x.
7. Executed the script and verified the output.

```
root@ip-172-31-26-233:/opt/home/scripts
[root@ip-172-31-26-233 scripts]# nano calculator.bash
[root@ip-172-31-26-233 scripts]# chmod +x calculator.bash
[root@ip-172-31-26-233 scripts]# ./calculator.bash
=====
CALCULATOR
=====
Enter first number:
4
Enter second number:
5
Select an operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
3
Result: 4 * 5 = 20
[root@ip-172-31-26-233 scripts]# cat calculator.bash
#!/bin/bash
# Purpose: Calculator
# Date: 16/08/2026
# Modification: 16/08/2026
# Owner: Vamshi

echo "=====
echo "      CALCULATOR
echo "=====

echo "Enter first number:"
read num1

echo "Enter second number:"
read num2

echo "Select an operation:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "5. Modulus"

read choice

case $choice in
    1)
        result=$((num1 + num2))
```

```
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "5. Modulus"

read choice

case $choice in
    1)
        result=$((num1 + num2))
        echo "Result: $num1 + $num2 = $result"
        ;;
    2)
        result=$((num1 - num2))
        echo "Result: $num1 - $num2 = $result"
        ;;
    3)
        result=$((num1 * num2))
        echo "Result: $num1 * $num2 = $result"
        ;;
    4)
        if [ "$num2" -eq 0 ]; then
            echo "Error: Cannot divide by zero."
        else
            result=$((num1 / num2))
            echo "Result: $num1 / $num2 = $result"
        fi
        ;;
    5)
        if [ "$num2" -eq 0 ]; then
            echo "Error: Cannot perform modulus by zero."
        else
            result=$((num1 % num2))
            echo "Result: $num1 % $num2 = $result"
        fi
        ;;
    *)
        echo "Invalid choice."
        ;;
esac
[root@ip-172-31-26-233 scripts]# |
```

Conclusion:

Successfully created a Bash calculator for basic arithmetic operations.

Task 8: Create a bash script to check if a directory exists, if not then create a directory.

Objective:

Create a Bash script to check whether a directory exists and create it if required.

Purpose:

To automatically create a directory when it is not available.

Implementation:

1. Created check_directory.sh.
2. Used read to accept the directory path.
3. Used the -d condition to check whether the directory exists.
4. Used mkdir to create the directory if it does not exist.
5. Made the script executable using chmod +x.
6. Executed the script and verified the directory using ls.

```
[root@ip-172-31-26-233 scripts]# nano directory_check.bash
[root@ip-172-31-26-233 scripts]# cat directory_check.bash
#!/bin/bash
# Purpose: Checking for directory exists or not, create directory
#Date: 16/08/2026
#Modification: 16/08/2026
#Owner= vamshi

read -p "Enter directory name: " dir
if [ -d "$dir" ]
then
    echo "Directory already exists."
else
    echo "Directory does not exist."
    echo "Creating directory..."
    mkdir -p "$dir"
    if [ $? -eq 0 ]
    then
        echo "Directory created successfully."
    else
        echo "Failed to create directory."
        exit 1
    fi
fi

echo "Script execution completed."
[root@ip-172-31-26-233 scripts]# chmod 755 directory_check.bash
[root@ip-172-31-26-233 scripts]# ./directory_check.bash
Enter directory name: /opt/home/scripts/test
Directory does not exist.
Creating directory...
Directory created successfully.
Script execution completed.
[root@ip-172-31-26-233 scripts]# ls
backup_directory.bash  check_nginx.bash  directory_check.bash  file2.txt  file4.txt  install_nginx.bash  sample  sample-20260815-102059.tar.gz  test
check_directory.bash  create_files.bash  file1.txt             file3.txt  file5.txt  install_tomcat.bash  sample
```

Conclusion:

Successfully checked and created a directory automatically using Bash scripting.

Task 9: Create a bash script to delete the last 3 lines of a file.

Objective:

Create a Bash script to remove the last three lines from a file.

Purpose:

To automate file cleanup by removing unwanted trailing lines.

Implementation:

1. Created delete_last3.bash.
2. Specified the input file – employees.txt.
3. Used head -n -3 to remove the last three lines.
4. Stored the modified content in a temporary file.
5. Replaced the original file with the modified file.
6. Verified the file contents using cat.

```
root@ip-172-31-26-233:/opt/home/scripts
To github.com:vamshikumar-a/bash-scripting.git
61f40fb...: elias23 main -> main
[root@ip-172-31-26-233 scripts]# nano employees.txt
[root@ip-172-31-26-233 scripts]# cat employees.txt
Employee1
Employee2
Employee3
Employee4
Employee5
Employee6
Employee7
[root@ip-172-31-26-233 scripts]# nano delete_last3.bash
[root@ip-172-31-26-233 scripts]# cat delete_last3.bash
#!/bin/bash
# Purpose: Deleting last 3 lines of file
# Date: 16/08/2026
# Modification: 16/08/2026
# Owner: vamshi

read -p "Enter file name: " file
if [ -f "$file" ]
then
    head -n -3 "$file" > temp.txt
    if [ $? -eq 0 ]
    then
        mv temp.txt "$file"
        echo "Last 3 lines deleted successfully."
    else
        echo "Failed to delete the last 3 lines."
        rm -f temp.txt
        exit 1
    fi
else
    echo "File does not exist."
    exit 1
fi
[root@ip-172-31-26-233 scripts]# chmod 755 delete_last3.bash
[root@ip-172-31-26-233 scripts]# ./delete_last3.bash
Enter file name: employees.txt
Last 3 lines deleted successfully.
[root@ip-172-31-26-233 scripts]# cat employees.txt
Employee1
Employee2
Employee3
Employee4
[root@ip-172-31-26-233 scripts]# |
```

Conclusion:

Successfully deleted the last three lines from the specified file

Task 10: Bash script to monitor cpu and if it is more than 80% then send email notification.

Objective:

Create a Bash script to monitor CPU usage and send an email notification when usage exceeds 80%.

Purpose:

To identify high CPU utilization and provide an automatic alert.

Implementation:

1. Created cpu_monitor.bash.
2. Used top to obtain CPU usage.
3. Calculated the CPU utilization percentage.
4. Compared the usage with the 80% threshold.
5. Sent an email notification when CPU usage exceeded 80%.
6. Made the script executable using chmod +x.
7. Executed the script and verified the CPU usage.

```
root@ip-172-31-26-233:/opt/home/scripts
[root@ip-172-31-26-233 scripts]# nano cpu_monitor.bash
[root@ip-172-31-26-233 scripts]# cat cpu_monitor.bash
#!/bin/bash
# Purpose: Checking for cpu usage and more than 80 ,it will send email to user
#Date: 16/08/2026
#Modification: 16/08/2026
#Owner= vamshi

# CPU usage threshold
THRESHOLD=80

# Email address
EMAIL="vamshikumara1004@gmail.com"

# Get hostname
HOSTNAME=$(hostname)

# Calculate CPU usage
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

# Remove decimal part
CPU_USAGE=${CPU_USAGE%.*}

echo "Current CPU usage: ${CPU_USAGE}%"

# Check CPU usage
if [ "$CPU_USAGE" -ge "$THRESHOLD" ]; then
    SUBJECT="HIGH CPU USAGE ALERT - $HOSTNAME"
    MESSAGE="WARNING!

Server: $HOSTNAME
CPU Usage: ${CPU_USAGE}%

CPU usage has reached or exceeded the ${THRESHOLD}% threshold.
Please check the server."

    echo "$MESSAGE" | mail -s "$SUBJECT" "$EMAIL"

    if [ $? -eq 0 ]; then
        echo "Email notification sent successfully."
    else
        echo "Failed to send email notification."
        exit 1
    fi
fi
```

```
Please check the server."

    echo "$MESSAGE" | mail -s "$SUBJECT" "$EMAIL"

    if [ $? -eq 0 ]; then
        echo "Email notification sent successfully."
    else
        echo "Failed to send email notification."
        exit 1
    fi
fi

else

    echo "CPU usage is normal: ${CPU_USAGE}%"

fi
[root@ip-172-31-26-233 scripts]# chmod 755 cpu_monitor.bash
[root@ip-172-31-26-233 scripts]# ./cpu_monitor.bash
Current CPU usage: 3%
CPU usage is normal: 3%
[root@ip-172-31-26-233 scripts]#
```

Conclusion:

Successfully created a CPU monitoring script with email notification for high CPU usage.

Task 11: Bash script to monitor disk space and if it is more than 80% then send email notification.

Objective:

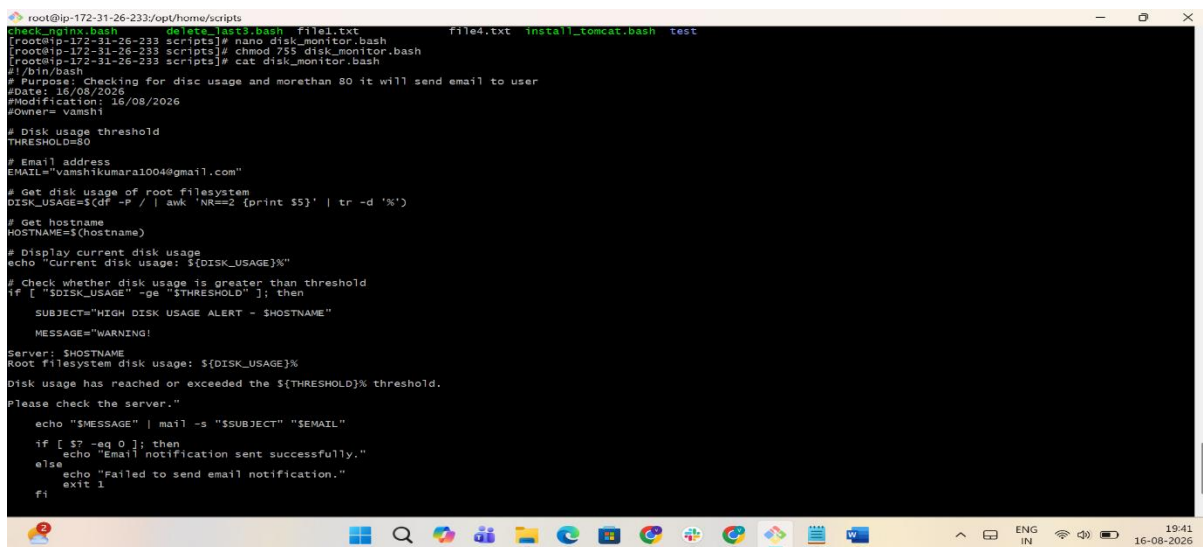
Create a Bash script to monitor disk usage and send an email notification when it exceeds 80%.

Purpose:

To prevent disk space issues by providing an automatic alert.

Implementation:

1. Created disk_monitor.bash.
2. Used df to check disk utilization.
3. Extracted the usage percentage.
4. Compared the usage with the 80% threshold.
5. Sent an email notification when usage exceeded 80%.
6. Made the script executable using chmod +x.
7. Executed the script and verified disk usage.



```
root@ip-172-31-26-233:/opt/home/scripts# nano disk_monitor.bash
root@ip-172-31-26-233:/opt/home/scripts# chmod 755 disk_monitor.bash
root@ip-172-31-26-233:/opt/home/scripts# cat disk_monitor.bash
#!/bin/bash
# Purpose: Checking for disc usage and morethan 80 it will send email to user
#Date: 16/08/2026
#Modification: 16/08/2026
#Owner= vamshi

# Disk usage threshold
THRESHOLD=80

# Email address
EMAIL="Vamshikumara1004@gmail.com"

# Get disk usage of root filesystem
DISK_USAGE=$(df -P / | awk 'NR==2 {print $5}' | tr -d '%')

# Get hostname
HOSTNAME=$(hostname)

# Display current disk usage
echo "Current disk usage: ${DISK_USAGE}%"

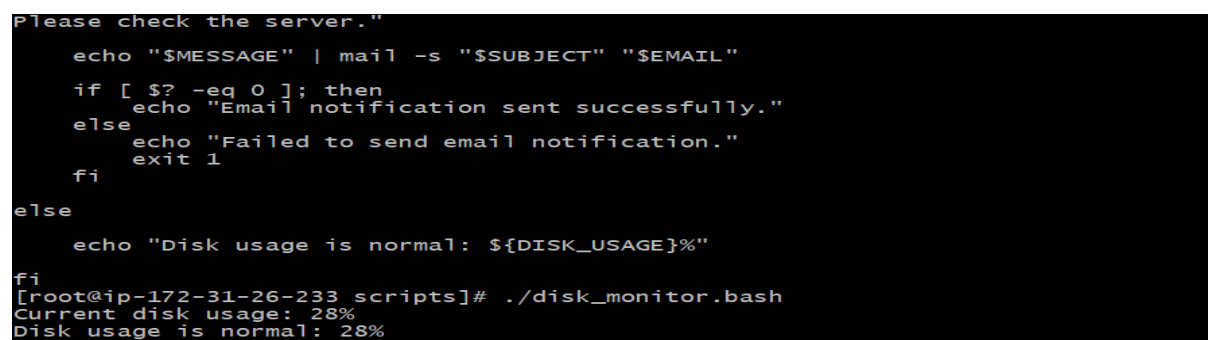
# Check whether disk usage is greater than threshold
if [ "$DISK_USAGE" -ge "$THRESHOLD" ]; then
    SUBJECT="HIGH DISK USAGE ALERT - $HOSTNAME"
    MESSAGE="WARNING!"

    Server: $HOSTNAME
    Root filesystem disk usage: ${DISK_USAGE}%
    Disk usage has reached or exceeded the ${THRESHOLD}% threshold.
    Please check the server."

    echo "$MESSAGE" | mail -s "$SUBJECT" "$EMAIL"
    if [ $? -eq 0 ]; then
        echo "Email notification sent successfully."
    else
        echo "Failed to send email notification."
        exit 1
    fi
fi

else
    echo "Disk usage is normal: ${DISK_USAGE}%"
fi

[root@ip-172-31-26-233:/opt/home/scripts]# ./disk_monitor.bash
Current disk usage: 28%
Disk usage is normal: 28%
```



```
Please check the server."

echo "$MESSAGE" | mail -s "$SUBJECT" "$EMAIL"
if [ $? -eq 0 ]; then
    echo "Email notification sent successfully."
else
    echo "Failed to send email notification."
    exit 1
fi
else
    echo "Disk usage is normal: ${DISK_USAGE}%"
fi
[root@ip-172-31-26-233:/opt/home/scripts]# ./disk_monitor.bash
Current disk usage: 28%
Disk usage is normal: 28%
```

Conclusion:

Successfully created a disk monitoring script with email notification for high disk utilization.

Task 12: Bash script to monitor memory and if it is more than 80% then send email notification.

Objective:

Create a Bash script to monitor memory usage and send an email notification when it exceeds 80%.

Purpose:

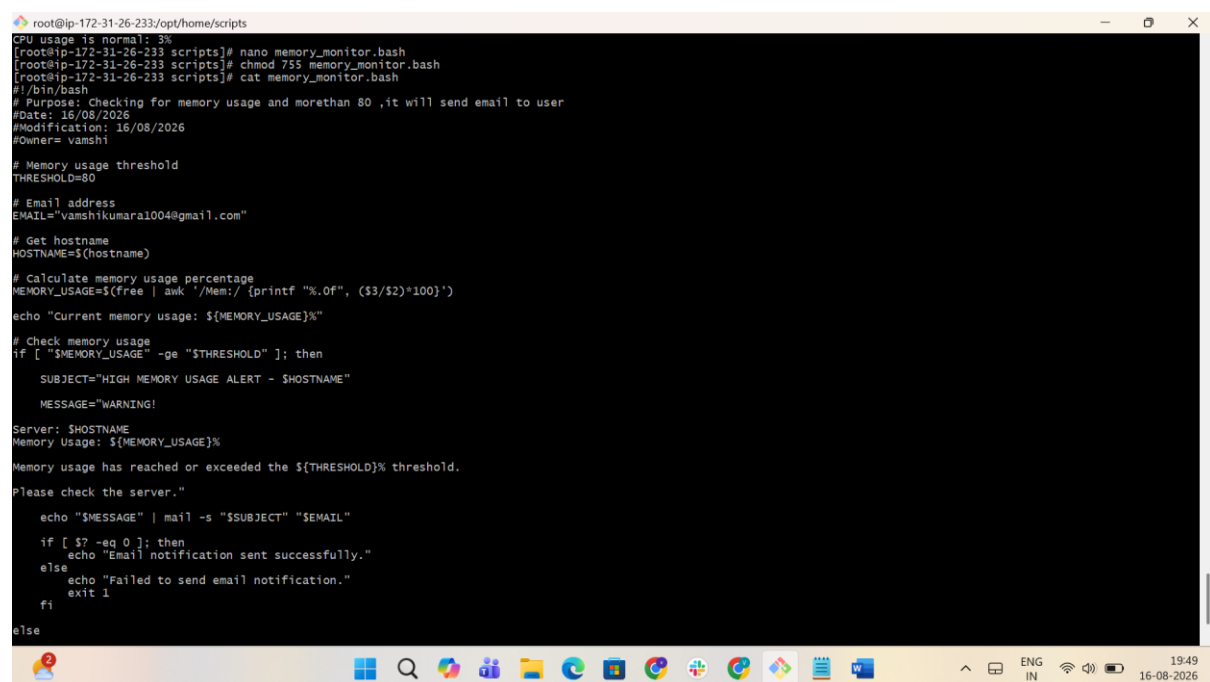
To identify high memory utilization and automatically notify the administrator.

Implementation:

1. Created memory_monitor.bash.
2. Used free to check memory utilization.
3. Calculated the memory usage percentage.
4. Compared the usage with the 80% threshold.
5. Sent an email notification when usage exceeded 80%.
6. Made the script executable using chmod +x.
7. Executed the script and verified memory usage.

Conclusion:

Successfully created a memory monitoring script with email notification for high memory utilization.

A screenshot of a terminal window titled 'root@ip-172-31-26-233:/opt/home/scripts'. The terminal shows the execution of a script named 'memory_monitor.bash'. The script's content is displayed, including comments about its purpose (checking memory usage and sending email notifications), a threshold of 80%, an email address 'vamshikumara1004@gmail.com', and the logic to calculate memory usage percentage and send an alert if it exceeds the threshold. The script also prints the current memory usage and the threshold. The terminal output shows the script running successfully, printing the current memory usage and the threshold, and sending an email notification. The Windows taskbar is visible at the bottom of the screen.

```
root@ip-172-31-26-233:/opt/home/scripts
CPU usage is normal: 3%
[root@ip-172-31-26-233 scripts]# nano memory_monitor.bash
[root@ip-172-31-26-233 scripts]# chmod 755 memory_monitor.bash
[root@ip-172-31-26-233 scripts]# cat memory_monitor.bash
#!/bin/bash
# Purpose: Checking for memory usage and morethan 80 ,it will send email to user
#Date: 16/08/2026
#Modification: 16/08/2026
#Owner= vamshi

# Memory usage threshold
THRESHOLD=80

# Email address
EMAIL="vamshikumara1004@gmail.com"

# Get hostname
HOSTNAME=$(hostname)

# Calculate memory usage percentage
MEMORY_USAGE=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')
echo "Current memory usage: ${MEMORY_USAGE}%"

# Check memory usage
if [ "${MEMORY_USAGE}" -ge "${THRESHOLD}" ]; then
    SUBJECT="HIGH MEMORY USAGE ALERT - $HOSTNAME"
    MESSAGE="WARNING!

Server: $HOSTNAME
Memory Usage: ${MEMORY_USAGE}%

Memory usage has reached or exceeded the ${THRESHOLD}% threshold.
Please check the server."

    echo "$MESSAGE" | mail -s "$SUBJECT" "$EMAIL"
    if [ $? -eq 0 ]; then
        echo "Email notification sent successfully."
    else
        echo "Failed to send email notification."
        exit 1
    fi
fi
else
```

```

        exit 1
    fi
else
    echo "Memory usage is normal: ${MEMORY_USAGE}%"
fi
[root@ip-172-31-26-233 scripts]# ./memory_monitor.bash
Current memory usage: 33%
Memory usage is normal: 33%

```

Exercise: Crontab Entries

1. April 5th Midnight
2. 5th of Every November, January, June if it is a Thursday
3. At 05 and 27th minutes of 9, 10, 11 hours every day
4. 34 minutes of 9th hour on 15th August
5. Every midnight
6. Every Weekend (Saturday night 11:59 PM)
7. After every reboot

```

[root@ip-172-31-26-233 scripts]# nano test.bash
You have new mail in /var/spool/mail/root
[root@ip-172-31-26-233 scripts]# ./test.bash
[root@ip-172-31-26-233 scripts]# crontab -l
* * * * * /opt/home/scripts/test.bash
[root@ip-172-31-26-233 scripts]# cat /opt/home/scripts/cron_test.log
Cron job executed at Sun Aug 16 14:42:52 UTC 2026
Cron job executed at Sun Aug 16 14:43:01 UTC 2026
Cron job executed at Sun Aug 16 14:44:01 UTC 2026
Cron job executed at Sun Aug 16 14:45:02 UTC 2026
[root@ip-172-31-26-233 scripts]#

```

```

[root@ip-172-31-26-233 scripts]# nano cron-entries.sh
[root@ip-172-31-26-233 scripts]# cat cron-entries.sh
# April 5 midnight
0 0 5 4 * /opt/home/scripts/test.bash

# 05 and 27 minutes of 9, 10, 11
5,27 9,10,11 * * * /opt/home/scripts/test.bash

# August 15 at 9:34 AM
34 9 15 8 * /opt/home/scripts/test.bash

# Every midnight
0 0 * * * /opt/home/scripts/test.bash

# Saturday 11:59 PM
59 23 * * 6 /opt/home/scripts/test.bash

# After reboot
@reboot /opt/home/scripts/test.bash
[root@ip-172-31-26-233 scripts]#

```